

WCL Log Analyzer

A modular Python toolkit for pulling Warcraft Logs reports and turning them into raid-useful analysis -- healing, damage, deaths, cooldowns, consumables, gear, and avoidable-damage tracking.

This guide covers what's built so far, how each module works, and how to run it yourself -- including install steps and where to get your API credentials.

Contents

- 1. Overview & Architecture
- 2. Installation & Setup
- 3. Getting Warcraft Logs API Credentials
- 4. Running the Example Script
- 5. Module Reference
 - 5.1 wcl_api.py -- API client
 - 5.2 data_models.py -- shared types
 - 5.3 log_parser.py -- raw JSON to models
 - 5.4 roster.py -- players vs. pets/NPCs
 - 5.5 death_analyzer.py
 - 5.6 healing_analyzer.py
 - 5.7 damage_analyzer.py (damage taken)
 - 5.8 damage_done_analyzer.py (DPS)
 - 5.9 cooldown_analyzer.py
 - 5.10 consumable_data.py & consumables_analyzer.py
 - 5.11 gear_analyzer.py
 - 5.12 avoidable_damage_data.py, boss_mechanics_config.py & avoidable_damage_analyzer.py
 - 5.13 ability_explorer.py (in progress)
- 6. Running the Tests
- 7. Known Limitations
- 8. What's Not Built Yet

1. Overview & Architecture

The project pulls combat log data from Warcraft Logs and turns it into readable, actionable analysis. It's built as small, independent Python modules rather than one large script, so each piece can be developed, tested, and understood on its own.

Every analyzer module is a **pure function**: it takes already-parsed fight data in and returns results out, with no network calls of its own. This means every analyzer (except `wcl_api.py` itself, whose entire job *is* the network call) can be tested offline with small hand-built fixture data -- no live report or API credentials required to run the test suite.

The data flow looks like this:

```
wcl_api.py (talks to the Warcraft Logs API)
| raw JSON: fights, master data (actors/abilities), events
v
log_parser.py (raw JSON -> this app's own data models)
| a ParsedFight: Fight + Actors + Abilities + Events + CombatantInfo
v
roster.py (used by every analyzer below to tell real players
          apart from NPCs/pets, and fold pet activity into owners)
|
v
analyzer modules (death, healing, damage taken/done, cooldowns,
                  consumables, gear, avoidable damage)
| each returns a list of small dataclass "report" objects
v
(not built yet) report module -- render everything into one document
(not built yet) main -- orchestrates the whole chain end-to-end
```

Right now, `example_usage.py` plays the role "main" will eventually play: it calls every module in sequence against one real report/fight and prints the results. See Section 4.

Project file layout

Module	Purpose
wcl_api.py	Talks to the Warcraft Logs v2 API. The only module that touches the network.
data_models.py	Shared dataclasses used by every module downstream of the API.
log_parser.py	Converts raw WCL JSON into the data models above.
roster.py	Identifies real players vs. NPCs/pets; folds pet activity into owners.
death_analyzer.py	Per-death reports with damage/healing context leading up to each one.
healing_analyzer.py	Per-healer and per-target healing summaries (HPS, overheal, breakdown).
damage_analyzer.py	Per-player damage-TAKEN summaries (DTPS) and biggest-hits detection.
damage_done_analyzer.py	Per-player damage-DONE summaries (DPS to the boss).
cooldown_analyzer.py	Tracks usage/efficiency of caller-specified cooldown abilities.
consumable_data.py	Reference list of Midnight consumable item names, by category.
consumables_analyzer.py	Per-player consumable coverage, plus special Vantus Rune logic.
gear_analyzer.py	Gear quality tier and enchant/gem presence per player.
avoidable_damage_data.py	Schema (dataclasses) for boss avoidable-mechanic config.
boss_mechanics_config.py	The actual per-boss mechanic data -- edit this file, not the analyzer.
avoidable_damage_analyzer.py	Counts avoidable hits per player per mechanic, for a specific boss.
ability_explorer.py	(In progress) Helper for identifying new avoidable mechanics from a log.
example_usage.py	Runs the whole chain against a real report -- the current "main" stand-in.
requirements.txt	Python package dependencies (currently just requests).

2. Installation & Setup

Requirements

- **Python 3.10 or newer.** The code uses modern type-hint syntax (e.g. `int | None`) that requires 3.10+.
- **pip** (comes with Python).
- No database, no external services -- everything runs as plain scripts.

Install steps

1. Put all the project's .py files in one folder.
2. Install the one dependency:

```
pip install -r requirements.txt
```

```
# or directly:  
pip install requests
```

That's the entire install. There's no build step, no compiled extensions, and no other services to stand up.

Verifying the install

Run the offline test suite -- this needs no API credentials at all:

```
python -m unittest discover -p "test_*.py" -v
```

If that finishes with OK at the bottom, the install is good and every analyzer module works correctly against its own test fixtures.

3. Getting Warcraft Logs API Credentials

The API client (`wcl_api.py`) authenticates using OAuth2 client-credentials against the Warcraft Logs v2 API. You need a client ID and client secret from your own Warcraft Logs API client.

Steps:

- Log in at warcraftlogs.com.
- Go to your account's API client management page (under your profile / API Clients).
- Create a new client (a name and any redirect URL are enough -- this app uses the client-credentials flow, not a user login flow, so the redirect URL isn't actually used).
- Copy the generated **Client ID** and **Client Secret**.

Setting credentials as environment variables

Windows (cmd.exe):

```
set WCL_CLIENT_ID=your-client-id-here
set WCL_CLIENT_SECRET=your-client-secret-here
```

Note: set only lasts for that terminal session -- you'll need to set it again in a new window. Use `$env:WCL_CLIENT_ID="..."` (with quotes) instead if you're using PowerShell rather than cmd.exe.

macOS / Linux (bash/zsh):

```
export WCL_CLIENT_ID=your-client-id-here
```

Keep these secret -- treat them like a password. If a client ID/secret pair is ever accidentally shared (e.g. pasted in a screenshot), regenerate it from the Warcraft Logs API client page.

4. Running the Example Script

`example_usage.py` is the current end-to-end demonstration: it fetches one report, picks the first fight, runs every analyzer against it, and prints the results. This stands in for the "main" orchestrator, which hasn't been built yet.

Usage:

```
python example_usage.py <report_code>
```

The report code is the part of a Warcraft Logs URL after `/reports/` -- for example, given `https://www.warcraftlogs.com/reports/CYZyV7DdghtNGb9a`, the report code is `CYZyV7DdghtNGb9a`.

What it prints, in order:

- The report title/zone and a list of its first 5 fights.
- A parsed summary of the first fight (name, kill/wipe, duration).
- The first 10 parsed events, with names resolved.
- Death analysis (who died, to what, damage/healing context beforehand).
- Healing analysis (per-healer HPS, overheal %, ability breakdown).
- Damage-taken analysis (per-player DTPS) and the biggest hits of the fight.
- Damage-done analysis (per-player DPS to the boss).
- Cooldown usage (for a small example set of raid cooldowns -- edit this list in the script to match your actual roster).
- Consumables analysis (who's missing what, plus the Vantus Rune check).
- Gear check (quality tier, missing enchant slots, gem counts).
- Avoidable damage analysis (empty until you fill in `boss_mechanics_config.py` for that boss).

5. Module Reference

Each entry below covers what the module does, its main functions/classes, and a short usage example. All analyzer modules take a `ParsedFight` (from `log_parser.py`) as their main input.

5.1 `wcl_api.py` -- API client

The only module that talks to the network. Handles OAuth2 authentication (and automatic token refresh), wraps the Warcraft Logs v2 GraphQL API, transparently paginates the events endpoint, and tracks API rate-limit usage. Returns raw WCL JSON only -- no interpretation happens here, that's `log_parser.py`'s job.

Key methods on `WCLClient`:

- `get_report_fights(report_code)` -- report metadata and the list of fights.
- `get_report_master_data(report_code)` -- report-wide actor and ability ID-to-name lookups.
- `get_report_events(report_code, fight_id, event_type, ...)` -- raw events of ONE type for one fight. `event_type` is required -- WCL silently returns null events if it's left unset, despite the field being nullable in their schema.
- `get_report_events_multi(report_code, fight_id, event_types)` -- fetches several event types for the same fight in one call, reusing the fight's time window across all of them.

Valid event types (`EVENT_DATA_TYPES`):

Bufs, Casts, CombatantInfo, DamageDone, DamageTaken, Deaths, Debuffs, Dispel, Healing, Interrupts, Resources, Summons, Threat

Example:

```
from wcl_api import WCLClient

client = WCLClient(client_id, client_secret)
report = client.get_report_fights("CYZyV7DdghtNGb9a")
fight_id = report["fights"][0]["id"]

master_data = client.get_report_master_data("CYZyV7DdghtNGb9a")
events = client.get_report_events_multi(
    "CYZyV7DdghtNGb9a", fight_id,
    event_types=["Casts", "Healing", "DamageTaken", "Deaths"],
)
```

5.2 `data_models.py` -- shared types

The dataclasses every other module (except `wcl_api.py`) works with:

- **Ability** -- id, name, type.
- **Actor** -- id, name, type (Player/NPC/Pet), subtype (class), server, and `owner_id` for pets.
- **Fight** -- id, name, difficulty, kill/wipe, start/end time, `encounter_id`, and `friendly_player_ids` (the ACTUAL roster present in this specific pull).

- **Event** -- a normalized combat log event: timestamp, type, source/target id+name, ability id+name, amount/overheal/absorbed/overkill, plus the original raw dict for anything not promoted to a named field.
- **GearItem** -- one equipped item: slot, item id, quality tier, item level, enchant id, gem ids. IDs only -- no names (no item database).
- **CombatantInfoSnapshot** -- a player's gear and active auras, captured at the START of a pull (catches flasks/food/runes popped before the pull began).
- **ParsedFight** -- the bundle every analyzer receives: fight, actors, abilities, events, and combatant_info.

5.3 log_parser.py -- raw JSON to models

Pure transformation, no network calls. Converts wcl_api's raw dicts into the data_models above.

- `parse_fights(raw_report) / parse_fight(raw_fight)` -- Fight objects.
- `parse_actors / parse_abilities(raw_master_data)` -- ID lookup dicts.
- `parse_events(raw_events_by_type, actors, abilities)` -- merges every event type into one chronologically sorted list, resolving IDs to names.
- `parse_combatant_info(raw_combatant_info_events, abilities)` -- separate path for CombatantInfo, since its gear/auras shape doesn't fit the generic Event model.
- `parse_fight_bundle(raw_fight, raw_master_data, raw_events_by_type)` -- convenience one-call entry point that does all of the above and returns a ready-to-use ParsedFight.

Example:

```
import log_parser

parsed = log_parser.parse_fight_bundle(raw_fight, master_data, events_by_type)
print(parsed.fight.name, parsed.fight.kill, len(parsed.events))
```

5.4 roster.py -- players vs. pets/NPCs

Shared helper used by every analyzer that needs to tell real players apart from NPCs, and fold pet/summon activity (Wild Imps, hunter pets, Dancing Rune Weapon, etc.) back onto the owning player.

- `get_players(actors)` -- filters to Player-type actors only.
- `get_player_roster(parsed_fight)` -- same, but report-wide (includes anyone who appeared anywhere in the whole report, e.g. subs from other pulls).
- `get_fight_roster(parsed_fight)` -- the ACCURATE roster for one specific pull, using `Fight.friendly_player_ids`. Prefer this one.
- `is_player(actor_id, actors)` -- boolean check.
- `resolve_to_player(actor_id, actors)` -- resolves a pet to its owner's Actor, or None for NPCs/unresolvable pets.
- `resolve_to_player_id_name(actor_id, fallback_name, actors)` -- same, but returns an (id, name) tuple with a fallback.

5.5 death_analyzer.py

Reports every player death (pet deaths are excluded) with the damage taken and healing received in a configurable window (default 5 seconds) immediately before it -- enough context to tell "unhealable spike" apart from "healer missed it."

- `analyze_deaths(parsed_fight, context_window_ms=5000)` -> list of `DeathReport` (victim, killing ability, damage/healing instances before death, with helper properties for totals).
- `summarize_wipe(parsed_fight, death_reports)` -> a short readable summary string.

Example:

```
import death_analyzer

reports = death_analyzer.analyze_deaths(parsed)
print(death_analyzer.summarize_wipe(parsed, reports))
```

5.6 healing_analyzer.py

Per-healer output (effective healing, overheal %, HPS, breakdown by spell) and per-target output (who received how much, from whom). Pet healing is folded into the owning player via `roster.py`.

Important: WCL's `amount` field on a heal event is already the **EFFECTIVE** (net) healing -- `overheal` is a separate, additive figure on top of it, not a portion carved out of it. Effective healing is therefore just `amount`, never `amount - overheal` (that produces negative HPS and >100% overheal, which was an early bug here).

- `analyze_healing(parsed_fight)` -> list of `HealerSummary`, sorted by effective healing descending.
- `analyze_healing_received(parsed_fight)` -> list of `TargetHealingSummary`.
- `summarize_healing(parsed_fight, summaries)` -> readable HPS-ranked summary string.

5.7 damage_analyzer.py (damage taken)

Damage TAKEN by players (bosses/adds hitting the raid) -- as opposed to damage_done_analyzer.py, which covers the reverse direction. Filters out damage landing on pets, so a pet absorbing a hit doesn't look like a raid member taking damage.

- `analyze_damage_taken(parsed_fight)` -> per-player DamageTakenSummary (DTPS, breakdown by ability and by source).
- `analyze_damage_sources(parsed_fight)` -> per-source DamageSourceSummary (which boss/add is dealing the most damage, and to whom).
- `get_biggest_hits(parsed_fight, top_n=5)` -> the single largest damage-taken events, across all players.
- `summarize_damage_taken(...)` -> readable DTPS-ranked summary.

5.8 damage_done_analyzer.py (DPS)

The "who's dealing damage to the boss" view. Pet damage (Wild Imps, hunter pets, Dancing Rune Weapon, etc.) is folded into the owning player via `roster.py`; damage from sources that aren't players and can't be resolved to an owner (bosses, adds hitting each other) is excluded entirely -- this module is specifically a player DPS view.

- `analyze_damage_done(parsed_fight)` -> per-player DamageDoneSummary (DPS, breakdown by ability, breakdown by target hit).
- `summarize_damage_done(...)` -> readable DPS-ranked summary.

5.9 cooldown_analyzer.py

WCL doesn't know which abilities ARE cooldowns or what their cooldown duration is, so this module takes a caller-supplied list of `CooldownDefinition` (ability name, cooldown duration in seconds, category) rather than hardcoding any spec's kit.

- `CooldownDefinition(ability_name, cooldown_seconds, category=None)` -- what you're tracking.
- `analyze_cooldown_usage(parsed_fight, definitions)` -> per-player CooldownUsage: every cast timestamp, gaps between casts, and an `efficiency()` score against a theoretical max (fight length / cooldown -- an optimistic upper bound, not a hard target).
- `summarize_cooldown_usage(...)` -> readable casts-vs-max summary.

Example:

```
from cooldown_analyzer import CooldownDefinition, analyze_cooldown_usage

defs = [CooldownDefinition("Revival", 180, category="raid_cd")]
usages = analyze_cooldown_usage(parsed, defs)
```

5.10 consumable_data.py & consumables_analyzer.py

`consumable_data.py` holds reference item-name lists (health potions, combat potions, oils + Hunter ammo, food, Healthstone, and the optional Void-Touched Augment Rune), sourced from Method.gg's Midnight consumables guide. Names are a first pass -- verify against a real log, since WCL's combat-log spell name doesn't always match the item name exactly.

`consumables_analyzer.py` detects usage two ways, combined: `CombatantInfo` auras at pull start (catches anything popped BEFORE the pull began, like food/flasks) and Casts events during the fight (catches reactive usage, like a health potion mid-fight).

- `analyze_consumables(parsed_fight, definitions)` -> per-player `PlayerConsumables`, reporting the SPECIFIC item used per category (not just yes/no).
- `check_vantus_rune(parsed_fight, vantus_rune_name)` -> special handling: Vantus Rune is once-per-week, so "missing" is only flagged when at least half the roster already has it that fight (the signal the raid IS using it that week).
- `summarize_consumables(...)` -> readable missing-items summary, with an optional Vantus Rune section.

5.11 gear_analyzer.py

Reports gear quality and enchant/gem coverage per player, from `CombatantInfo` snapshots.

Limitation: this app has no item database, so enchants and gems are reported as present/missing plus raw IDs -- never by name (no "Enchant Boots - Defender's March", just "has an enchant" or not). Off-hand is excluded from the "missing enchant" check since it might be a shield, and we have no item-type data to know. Gems are reported as a raw count, not "X of Y sockets," since socket counts aren't known without an item database either.

- `analyze_gear(parsed_fight)` -> per-player `PlayerGearReport` (average item level, lowest quality tier, total gems, missing enchant slots).
- `summarize_gear(reports)` -> readable summary.

5.12 avoidable_damage_data.py, boss_mechanics_config.py & avoidable_damage_analyzer.py

WCL has no concept of "avoidable" -- a boss Cleave and a fire patch tick both just look like `DamageTaken` events. This has to be supplied by hand, boss by boss, split across three files:

- **avoidable_damage_data.py** -- just the schema: `AvoidableMechanic` (ability name, category, `track_via`) and `EncounterConfig` (`encounter_id`, name, list of mechanics).
- **boss_mechanics_config.py** -- the actual data. This is the file you edit to add a boss -- it contains the `ENCOUNTERS` dict plus three worked examples in comments (a ground-effect, a stacking debuff, and a second boss with cone/chase mechanics).
- **avoidable_damage_analyzer.py** -- matches the current fight to its config via `encounter_id`, then counts hits per mechanic per player.

track_via controls detection: "damage" counts DamageTaken events (use for anything that deals direct damage), "debuff" counts fresh Debuffs applications, not refreshes/ticks (use for stacking effects that punish later rather than on landing).

v1 scope: only "always avoidable" mechanics are supported -- things that are a mistake no matter who they hit (fire patches, cone breaths, chase effects). Intentional "soak" mechanics, where someone is SUPPOSED to get hit, need real assignment-aware logic that isn't built yet -- leave those out of the config for now.

- get_encounter_config(encounter_id, configs) -> the matching EncounterConfig, or None if that boss isn't configured yet.
- analyze_avoidable_damage(parsed_fight, configs) -> per-player PlayerAvoidableDamage (hit count and total damage per mechanic). Returns an empty list if the boss has no config.
- summarize_avoidable_damage(...) -> readable summary, or a "no config for this encounter yet" message.

Example -- adding a new boss:

```
# in boss_mechanics_config.py:
ENCOUNTERS[1234] = EncounterConfig(
    encounter_id=1234,
    encounter_name="Nek'zali the Soulcoiler",
    mechanics=[
        AvoidableMechanic("Corrupted Ground", category="ground_effect",
                           track_via=("damage",)),
    ],
)
```

5.13 ability_explorer.py (in progress)

A configuration-authoring aid, not part of the normal analysis pipeline: given a fight, it summarizes every ability used by NPCs (bosses/adds -- never players or pets) to help you decide what belongs in boss_mechanics_config.py. For each ability it reports: how often it happened, how many distinct players it hit, damage numbers (total/avg/max), when it first/last appeared (raw time and % into the fight), and whether it's ever been a killing blow.

Status: the analysis engine (analyze_boss_abilities and format_ability_table) is written but not yet covered by tests, and the interactive CLI wrapper for picking a report and fight hasn't been built yet. Not wired into example_usage.py. Treat this section as a preview of what's coming, not a finished tool.

6. Running the Tests

Every analyzer module (everything except `wcl_api.py`) has an offline unit test file using small hand-built fixture data -- no network access or API credentials required.

Run everything:

```
python -m unittest discover -p "test_*.py" -v
```

Run one module's tests:

```
python test_healing_analyzer.py  
# or  
python -m unittest test_healing_analyzer.py -v
```

As of this guide, the full suite is 122 tests, all passing. (`ability_explorer.py`, still in progress, has no test file yet.)

7. Known Limitations

- **No item database.** Enchants, gems, and items are only ever reported by numeric ID and WCL's own quality tier -- never by human-readable name. Getting names would require integrating an external item database, which isn't wired up.
- **Consumable/food names are first-pass.** consumable_data.py's names come from a community guide, not verified combat-log spell names for this content. Expect to correct the odd entry once tested against a real log.
- **No true "phase" data.** WCL doesn't expose phase transitions for brand-new content automatically -- ability_explorer.py uses "% of fight duration" as an honest proxy instead.
- **Avoidable damage is v1: no soak mechanics.** Only "always a mistake" mechanics are supported. A mechanic where someone is SUPPOSED to get hit needs real assignment-aware logic that doesn't exist yet.
- **Gear socket counts are unknown.** gear_analyzer.py can say "N gems found" but not "missing 1 of 3 sockets," since item socket counts aren't available without an item database.
- **masterData is report-wide, not per-fight.** This is why roster.get_fight_roster() exists and should be preferred over get_player_roster() when you specifically care about who was in one pull.

8. What's Not Built Yet

- **report module** -- something that takes every analyzer's output and renders it into one combined document/output, instead of example_usage.py's plain sequential print statements.
- **main orchestrator** -- thin glue that drives the whole chain (parser -> analyzers -> report) for real use, instead of example_usage.py standing in for it.
- **ability_explorer.py CLI** -- the interactive/CLI wrapper for picking a report and fight, plus its test file (see Section 5.13).
- **Soak-aware avoidable damage (v2)** -- flagging mechanics where hitting the WRONG number of people (or the wrong person) is the mistake, not just "was anyone hit at all."

End of guide. This reflects the project as of the modules listed in Section 5 -- update it as new modules are added.